Worker Isolation: Responsibility and Resource Table

Branch `fix/worker-isolation`, base `35884ee`. The feature is gated by `WORKER_ISOLATION_ENABLED`, which defaults to **off**. With the gate off, the worker runs the unchanged in-process path from `35884ee`. With it on, each job attempt gets one `child_process.fork`. The **parent** is the BullMQ supervisor and the **child** runs the existing `runAnalysisPipeline`.

File:line references point at the branch head. Each "Test" entry names a test in `server/lib/job-isolation.test.ts` (22/22 pass, see `TEST_RESULTS.md`).

1. Full DB pool inventory (gap A)

Every `pg.Pool` that a worker replica can create:

Pool	Process	Max connections	Created	Where enforced
Main pool, parent	worker parent	**4** (`min(WORKER_ISOLATION_PARENT_PG_RESERVE=4, PG_POOL_MAX)`)	at import, lazy connect	`server/db.ts` L26-30 (cap applied only when role=worker-parent and gate on), L35-37 (`new Pool`)
Usage pool, parent	worker parent	**2** (`PG_USAGE_POOL_MAX`)	lazily, on first usage flush	`server/lib/llm-usage.ts` L308-315
Main pool, child	each job child	**4** = `floor((PG_POOL_MAX 20 − 4) / WORKER_CONCURRENCY 4)`	lazy; 0 connections at startup (F1)	`job-isolation.ts` `childPgBudget` L42, fork env `PG_POOL_MAX` L157; fork refused if budget < 1 (L134)
Usage pool, child	each job child	**0**: never created. Usage rows go to the parent over IPC	never	`llm-usage.ts` L344 (`isIsolatedChild() → sendToParent`)
Redis / BullMQ	parent only	unchanged	parent only	child does not load `redis.ts`, `ioredis` or `bullmq` (F1)

Totals per replica	Main	Usage	All PG
Baseline 35884ee (gate off)	20	2	**22**
Isolation at `WORKER_CONCURRENCY=4` (deployed)	4 + 4×4 = 20	2 + 0 = 2	**22**
Isolation at `WORKER_CONCURRENCY=10`	4 + 10×1 = 14	2	**16** (C = floor(16/10) = 1)

The earlier version of this table said "20 = PG_POOL_MAX" and left out the usage pool. The correct total is **22 = 20 main + 2 usage**, the same as baseline. Test: **A1** checks the count from the real modules: `db.ts` `PG_POOL_MAX` evaluated in a worker-parent role, `childPgBudget()`, `llm-usage` `USAGE_POOL_MAX`, and `__usagePoolCreatedForTests()` = false in a child. **F1** checks `pool.totalCount = 0` in a booted child.

2. Deadlines (gap B)

The parent enforces both deadlines using its own timers. It does not wait for the child to cooperate.

Deadline	Source	Parent action	Classification
Pipeline	`PIPELINE_TIMEOUT_MS`, same env as `pipeline.ts` L100 (code default 540 s; production 35 min)	timer from spawn (`job-isolation.ts` L201) → onDeadline: `revokeAttempt(lc,"pipeline_timeout")` + **immediate SIGKILL** of every group	`{error:"Pipeline timed out after Ns", failureType:"timeout"}`, the same string as in-process `pipeline.ts`. Retriable, which is the existing policy. Company marked failed + `logProcessingError (recordPipelineTimeout)`
Job	`deadlineAt = claimTime + JOB_TIMEOUT_MS` (`worker.ts` L339; code default 600 s; production 40 min)	timer at `lc.deadlineAt` (L202) → `revokeAttempt(lc,"job_watchdog_timeout")` + immediate SIGKILL	`Error("Job watchdog timeout: isolated child killed at job deadline (...)")` → non-retriable

Cleanup grace never extends past either deadline: - The cancel/revoke grace (`WORKER_ISOLATION_KILL_GRACE_MS` = 10 s, then SIGTERM, then +5 s SIGKILL) is pre-empted by `onDeadline`, which kills immediately. - The post-report exit grace (`WORKER_ISOLATION_EXIT_GRACE_MS` = 2 s) is also cut off by a deadline (L189-192). - `revokeAttempt` runs on the parent at the deadline, so the parent lifecycle refuses dispatch. Child score writes are refused by the existing SQL fence (`deadlineAt` + `attempt_token`).

Tests: - **B1**: child blocked forever with a 30 s grace configured. Resolved at 2529 ms (limit 2500 ms) with “Pipeline timed out”, revoked `pipeline_timeout`. Child, grandchild and detached descendant were all gone at settle. - **B2**: child keeps dispatching every 10 ms and ignores revoke and SIGTERM. It is SIGKILLed at `deadlineAt`. Its last raw (unfenced) fixture write landed within the timer + kill latency: +1 ms in the final run (−7 ms and −6 ms in earlier runs; test tolerance 50 ms). Nothing was written after that. Classified as non-retriable watchdog. Real score writes are additionally refused by the SQL `deadlineAt` fence. - **D6**: a cancel was in progress when the deadline arrived. The child was killed at the deadline, not after the 30 s grace, and the result stayed “Cancelled”.

3. Termination and parent death (gap C)

Event	Mechanism	Test
Graceful parent exit	`process.on("exit")` SIGKILLs all live groups (`job-isolation.ts` L141)	n/a
Parent SIGKILL / OOM (the exit hook does not run)	child-side **watchdog `worker_thread`** (`job-child.ts` L35) polls `process.ppid` every 250 ms on its own thread, so it also works while the child’s main loop is blocked. When the parent is gone it SIGKILLs registered descendant groups and the child’s own group. `disconnect` does the same when the child’s loop is free (L41)	**C** (blocked): supervisor SIGKILLed → child, grandchild and detached descendant gone in 41 ms. **C** (idle): 10 ms
Detached descendants (Chromium uses `detached:true`)	the child sends pgroup with Chromium’s pid (`processor.ts` L1035 → `isolation-ipc.ts registerDescendantGroup`). The parent adds it to the child’s kill set (`job-isolation.ts` L265)	B1, C, E2 (exercised with a detached `sleep`, **not** a real Chromium)

4. Retry table (gap D): same outcome as production

`classifyIsolatedOutcome` (`job-isolation.ts` L70) is the single place that decides how an isolated attempt ended. The decision then goes through `worker.ts`'s existing policy. That policy now lives in `server/lib/retry-policy.ts` (`NON_RETRIABLE_ERRORS`/`isRetriableError` moved verbatim) and `fenceCancelledResult` (the existing `worker.ts` Cancelled→watchdog fence, extracted with no behaviour change; called at `worker.ts` L465).

Event	Isolated outcome	`worker.ts` result	Matches in-process?	Test
Admin cancel (batch cancelled)	`"Cancelled"`	`failJob("Cancelled (batch cancelled)")`, **no retry**	yes	D row 1, test 2
Forced kill after cancel (child ignored it)	`"Cancelled"` (killReason stays `cancel`)	cancellation, no retry	yes	D row 2, **D6**
Ownership lost (revoke `ownership_lost`)	`"Cancelled"`, batch not cancelled → fenced to `"Job watchdog timeout … (attempt revoked: ownership_lost)"`	non-retriable. `failJob` with `transitioned:false` is a stale no-op (`worker.ts` L581-584, L689), so there is **no stale transition and no duplicate retry**	yes	D row 3, test 3
Job watchdog deadline	`Error("Job watchdog timeout: …")`	**final** (non-retriable)	yes	D row 4, **B2**
Pipeline deadline	`{error:"Pipeline timed out after Ns"}`	**retriable** (existing policy, kept and verified)	yes, identical string	D row 5, **B1**
Crash (non-zero exit, no kill requested)	`Error("Isolated job child exited unexpectedly (code=1 …)")`	retriable up to `MAX_RETRY_ATTEMPTS`, non-NULL `last_error`	same handling as an in-process thrown error	D row 6, test 4
OOM / external SIGKILL	same as crash	retriable	same	D row 7
Child reported before any kill	exactly what it reported	unchanged	yes	D row 8

5. Permits and background work (gap E)

Rule	Mechanism	Test
A child cannot report success while its background work is alive	`result/error` only records the outcome (`onReported`, L223). Settlement happens on child **exit**. Exit grace is 2 s, then SIGKILL. On exit: kill groups → poll until gone (≤ 2 s) → `releaseAll` → settle (L289-304)	**E2/E3**: delivered at 3517 ms. At delivery the child, grandchild, held proc permit and live set were all gone/0
Cancelled queued IPC acquisitions are removed	`cancelAcquire` aborts the parent's LLM waiter (L255-259). A grant arriving after withdrawal/exit is handed straight back (L249). On exit, ungranted waits are aborted (L208)	**E1**: pending 0 after abort while 8 were held. **E1b**: child dies with 8 held + 1 queued → all returned
Permits are not released before the resource has stopped	`releaseAll` runs only after the child has exited and every registered group is gone (L295-300)	**E2/E3** `releasedWhileAlive=false`
Subprocess limit = baseline 6 per replica	`MAX_CONCURRENT_SUBPROC` default 6 (`processor.ts` L557, unchanged from 35884ee), brokered by the parent	**E4**: 3 children × 4 → max 6

Rule	Mechanism	Test
LLM 8 / browser 1 per replica	parent `llmSlotBroker` / `processorSlotBroker`	test 5: max 8 / 1

6. Child startup (gap F)

The child entry is `job-child.ts`. It never imports `worker-main.ts` (which is where `initializeDatabase`, `cleanupOnStartup` and `startReconciler` run, L79-98).

Test **F1** recorded the child's state at boot: - Handles at entry: `PipeWrap, Timeout`. - After importing `pipeline.ts`: `PipeWrap, Timeout`. The only handle is the IPC pipe (fd3), and there are no sockets. - `pg` totalCount 0, and no usage pool. - Server modules loaded: only `server/storage.ts` and `server/db.ts`. `db.ts` builds a lazy `Pool` object with no connection. - Not loaded: `redis.ts`, queue, `startup-cleanup`, reconciler, `worker*.ts`, `reconcile-resurrect`, `ioredis`, `bullmq`. `redis.ts` throws at import when `REDIS_URL` is unset, and the test unsets it.

7. What still runs on the parent (gap G)

"The parent never runs pipeline CPU" is true. It is narrower than "the parent cannot be blocked". Work still on the parent loop:

Parent work	Cost / bound	Can it block?
BullMQ consume + lock renewal, Redis cancel cache	network I/O	no CPU risk
claim, heartbeat (×4), ownership watch (×4, 30 s), cancel poll (500 ms per child)	short single SQL / timer	no
`fork()` per attempt	G1: max 2.6-9.1 ms across runs (7.6 ms in the final run)	synchronous, bounded
`structuredClone` of init opts (company/framework/ measures/BatchFetchState)	proportional to payload	yes, if payloads grow large
IPC handlers (acquire/release/usage/stage/pgroup)	G1: max ≤ 0.10 ms per message	no
permit broker, usage-buffer flush (usage pool 2)	O(1) / batched INSERT	no
BatchFetchState merge at job end	O(hosts)	small
terminal writes, retry, batch counters (unchanged `worker.ts`)	SQL	no
GC of the parent heap	small heap (no pipeline objects)	short pauses
host CPU saturation (4 children on 2 vCPU)	children run at **nice 10** (L162)	mitigated, not eliminated

Test **G1**: 4 CPU-bound children on this 2-CPU host gave parent max lag **10.4 ms**, forks 4, max fork 7.6 ms, 12 messages, max handler 0.10 ms (final run; earlier runs 4.5 / 4.6 ms lag).

Preserved unchanged: scoring rules, BM25, research and scoring code, `PG_POOL_MAX`, `PG_USAGE_POOL_MAX`, `JOB_TIMEOUT_MS`, `PIPELINE_TIMEOUT_MS`, `WORKER_CONCURRENCY`, `LLM_MAX_CONCURRENCY`, `MAX_CONCURRENT_BROWSER`, `MAX_CONCURRENT_SUBPROC`. The gate stays off by default.